

2020 年面向对象程序设计期末考试参考答案

一、选择题

1.

主答案: A/C

cout 不是 C++ 关键字, 它是标准库中的输出流对象名。statics 也不是关键字, 它只是把 static 拼错了。因此从严格语言角度看, A 和 C 都不是关键字; 若按出题意图, 通常会选 C, 因为它最容易和“语言关键字”混淆。

2.

答案: D

复制构造函数的标准形式应当是:

```
Demo(const Demo& rhs)
```

它没有返回值类型, 函数名与类名相同, 参数通常是同类对象的常量引用。

3.

答案: B

逐个判断:

1. hello 是 string, 所以 hello + ", world" 合法, 结果仍是 string, 再加 "!" 也合法。
2. "Hello" + world 也是合法的, 因为标准库提供了 const char* 与 string 的拼接形式。
3. "Hello" + ", world" 两边都是字符串字面量, 本质上都是指针, 不能直接做字符串拼接。
4. 同理, 三个字符串字面量直接相加也不成立。

所以有效的是 1) 和 2)。

4.

答案: C

非 const 引用形参不能绑定到 const 对象或 const 引用, 因为这样会带来“通过非常量引用修改常量对象”的风险。而 const 引用则更宽松, 既可以绑定常量, 也可以绑定非常量。

5.

答案: C

模板参数的具体类型是在**编译期实例化**时确定的, 不是等到程序运行时才确定。因此 C 错, 这一题考察的是模板与泛型函数的基本原理。

6.

答案: B

public 标签下的成员, 不仅类外可以访问, 类内当然也可以访问。

public、private、protected 本质上是在说明“访问权限”, 而不是把类内和类外严格二分开。

7.

答案: C (不确定)

通常按基础课程语境, 这四句话里 1)、2)、4) 可视为正确, 3) 错误, 因此共有 3 项正确, 选 C。

说明如下:

1. 构造函数确实用于定义对象如何初始化。
2. 构造函数不像普通成员函数那样通过对象直接调用, 创建对象时会自动触发。
3. 构造函数不一定非要写在 public 下, 某些设计里还会故意写成 private 或 protected。
4. 构造函数名与类名相同, 且没有返回值类型。

8.

答案: C

TIOBE 是一个 const string 数组, 因此数组元素本身不能被修改:

- A 试图修改常量字符串中的字符, 不行
- B 试图给常量数组元素重新赋值, 也不行
- D 试图排序常量数组, 排序过程中需要交换元素, 所以也不行
- C 只是把内容复制到另一个普通数组 Lan 中, 这是允许的

二、程序阅读题

1.

输出结果:

2 3 1 2 3

关键在于这一句:

```
iter = li.erase(iter);
```

erase(iter) 会删除当前结点, 并返回“被删结点后面的那个位置”。但 for 循环结尾还会再执行一次 iter++, 于是就会跳过紧接着的下一个元素。

原链表为:

1 2 3 1 1 2 3

删除过程大致如下:

1. 删除第一个 1 后，迭代器先回到 2
2. for 的自增又把它推进到 3，于是 2 被跳过
3. 后面删除中间那个 1 时，同样会跳过它后面的那个 1

因此最后剩下：

2 3 1 2 3

2.

输出结果：

8

这题同时考察了静态成员、复制控制和后置自增。

先看对象个数：

1. Store p1, p2; 创建两个对象，静态成员 cnt = 2
2. Store p3 = p2; 调用复制构造后，cnt = 3

再看每个对象内部 Item::cnt 的变化：

1. 刚构造时，p1.itm->cnt = 0, p2.itm->cnt = 0
2. p2 = p1; 中，执行 itm->cnt = p.itm->cnt++;
先把 p1.itm->cnt 的旧值 0 赋给 p2.itm->cnt，再把 p1.itm->cnt 加到 1
3. Store p3 = p2; 同理，p3.itm->cnt 得到旧值 0，而 p2.itm->cnt 变成 1

于是主函数输出的是：

```
p1.getItem() + p2.getItem() + p1.getcnt() + p2.getcnt()
= 1 + 1 + 3 + 3
= 8
```

3.

输出结果：

(4, 2)

先看对象初始化：

- p1(1, 2) 得到 (1, 2)
- p2 = 1 会调用构造函数 Point(int i = 0, int j = 0)，因此得到 (1, 0)
- p3 = 2 同理得到 (2, 0)

然后：

p1 + p2 = (1, 2) + (1, 0) = (2, 2)

再执行：

p3 += (2, 2)

所以 p3 变成:

(4, 2)

这一题顺便考察了单参数构造函数引发的隐式类型转换。

4.

输出结果:

10 0 15 25

这里要同时注意三件事: 非虚函数、虚函数、默认参数。

1. x.add(5)

add 不是虚函数, x 的静态类型是 X, 所以调用 X::add(5), 结果为 $5 + 5 = 10$

2. x.multiply(5)

multiply 是虚函数, 会动态绑定到 Y::multiply, 但默认参数仍按静态类型 X 取 $j = 0$, 因此结果为

$5 * 0 + 0 * 5 = 0$

3. y.add(5,5)

直接调用派生类版本, 结果是 $5 + 5 + 0 + 5 = 15$

4. y.multiply(5,5)

结果是 $5 * 5 + 0 * 5 = 25$

所以最终输出为 10 0 15 25。

三、程序填空题

1.

填空答案:

```
(1) (int s = 0, int e = 0) : start(s), end(e) {}
(2) return a.start < b.start
(3) intervals.empty()
(4) intervals.begin(), intervals.end(), compare_interval
(5) idx < intervals.size()
(6) !merged.empty() && interval.start <= merged.back().end
(7) merged.push_back(interval)
(8) const Interval& interval = merged[idx]
```

这段程序的核心思路是:

1. 先按区间左端点排序
2. 再从前到后扫描
3. 若当前区间和 merged 的最后一个区间有重叠, 就合并
4. 否则把当前区间直接放入结果

其中第 (6) 空最关键，它在判断 “当前区间能否与最后一个已合并区间继续合并”。

四、编程题

1. 模板函数 selectSort

这一题主要考察函数模板、迭代器和选择排序思想。区间写成 `[begin, end)`，说明函数最好直接按迭代器来写，这样可以适用于不同容器。

参考实现如下：

```
#include <algorithm>

template <class Iter>
void selectSort(Iter begin, Iter end)
{
    for (Iter i = begin; i != end; ++i) {
        Iter minPos = i;
        for (Iter j = i; j != end; ++j) {
            if (*j < *minPos)
                minPos = j;
        }
        if (minPos != i)
            std::swap(*i, *minPos);
    }
}
```

它的做法很直接：每一轮先在未排序部分找出最小元素，再与当前位置交换。因为写成了模板函数，所以只要元素支持 `<` 比较和交换，这个排序函数就能工作。

2. Company、Employee、Manager、Programmer

这一题主要考察继承、多态、虚函数、虚析构函数以及容器中保存基类指针。公司中同时有经理和程序员，因此最自然的做法是：

1. 用 Employee 作为基类
2. 用虚函数 salary() 统一计算收入接口
3. Company 中用 vector<Employee*> 保存雇员
4. 删除员工和析构公司时都要记得释放动态申请的对象

参考实现如下：

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

```

class Employee {
public:
    Employee(const string& name) : n(name) {}
    virtual ~Employee() {}

    string name() const { return n; }
    virtual int salary() const = 0;

protected:
    string n;
};

class Manager : public Employee {
public:
    Manager(const string& name, int h, int b)
        : Employee(name), hours(h), bonus(b) {}

    int salary() const {
        return hours * 200 + bonus;
    }

private:
    int hours;
    int bonus;
};

class Programmer : public Employee {
public:
    Programmer(const string& name, int h, int o)
        : Employee(name), hours(h), overtime(o) {}

    int salary() const {
        return hours * 150 + overtime * 300;
    }

private:
    int hours;
    int overtime;
};

class Company {
public:

```

```

~Company() {
    for (std::size_t i = 0; i < ep.size(); ++i)
        delete ep[i];
}

void AddEmployee(Employee* e) {
    ep.push_back(e);
}

void RemoveEmployee(string name) {
    for (vector<Employee*>::iterator it = ep.begin(); it != ep.end(); ++it) {
        if ((*it)->name() == name) {
            delete *it;
            ep.erase(it);
            break;
        }
    }
}

void Print() const {
    for (std::size_t i = 0; i < ep.size(); ++i)
        cout << ep[i]->name() << ":" << ep[i]->salary() << endl;
}

private:
    vector<Employee*> ep;
};

```

按题干给定的收入规则：

- 经理收入 = 工作时长 * 200 + bonus
- 程序员收入 = 工作时长 * 150 + overtime * 300

因此：

- Wang 的收入是 $20 * 200 + 500 = 4500$
- Zhao 的收入是 $30 * 150 + 0 * 300 = 4500$
- Lin 的收入按题干应为 $15 * 150 + 5 * 300 = 3750$

这里最后一项与题面给出的 3700 不一致。复习时应以题干中的工资规则为准，样例输出大概率是旧卷中的笔误。若考试现场严格要求“与样例一致”，则需要以监考说明或老师补充解释为准；若没有额外说明，写出符合题意的程序通常更稳妥。